

Smart Placement System — Project Documentation

Submitted to: [Teacher's Name / Department] **Submitted by:** Souvagyo **Project Title:** Smart Placement System **Tech Stack:** MongoDB · Express.js · React (Vite) · Node.js · Socket.IO · Firebase Auth · Tailwind CSS · Nodemailer

1. Introduction

The **Smart Placement System** is a full-stack web application that digitises and automates the campus placement workflow. It connects three roles — **Students**, **Placement Officers**, and **Admins** — through a single platform that handles job postings, applications, eligibility checks, profile management, real-time notifications, and analytics.

The system eliminates the manual effort of circulating job notices over WhatsApp/email, tracking Excel sheets of applicants, and verifying eligibility one student at a time. It also provides automated reminders before application deadlines so students never miss an opportunity.

2. Objectives

- Centralise all placement-related information in one place.
 - Allow officers to post jobs and shortlist/accept/reject candidates quickly.
 - Automatically check student eligibility (CGPA, 10th/12th %) against job requirements.
 - Notify students in real time about new jobs, status changes, and deadline reminders.
 - Give admins an oversight dashboard with placement statistics.
 - Provide a clear, role-based UI for each user type.
-

3. User Roles

Role	Description
Student	Registers, completes profile, views/ applies for jobs, tracks application status, receives notifications.
Placement Officer	Manages students (verify/create), posts jobs, reviews applications, updates statuses, views placement stats.

Role	Description
Admin	Manages officers, has full system access, sees the master dashboard.

4. Tech Stack & Architecture

Frontend - React 18 with Vite - React Router for client-side routing - Tailwind CSS for styling - lucide-react for icons - Firebase Authentication (Google Sign-In) - Socket.IO client for real-time updates - Axios for API calls

Backend - Node.js + Express.js REST API - MongoDB + Mongoose ODM - Socket.IO server (WebSockets) - Firebase Admin SDK (token verification) - Nodemailer for email notifications - Multer for file uploads (resumes, profile photos) - JWT for session tokens

Architecture Pattern: MERN (MongoDB – Express – React – Node) with WebSocket layer for live updates and role-based access control (RBAC) middleware.

5. Modules & Features Implemented

5.1 Authentication & User Management

- **Three login flows:** student self-registration, Firebase Google sign-in, and a dedicated admin/officer login.
- **OTP-based email verification** for student registration.
- **Officer-created student accounts:** when an officer adds a student, a temporary password is generated and shown once. The student must change it and complete their profile on first login (enforced via `mustChangePassword` and `isProfileComplete` flags in `/me`).
- **Password change** and **profile completion** flows.
- **JWT-based** protected routes with role middleware.
- **Active / Rejected / Suspended** account status handled by officers.

5.2 Student Profile

- Comprehensive profile fields: personal info (name, DOB, gender, address, Aadhar), family details, academic records (10th/12th %, CGPA, backlogs, graduation year), skills, resume upload.
- Skills stored as a tag array for easy matching.
- **Profile completeness check** before allowing job applications.

5.3 Job Management

- Officers can **create, edit, close, and delete** job postings.

- Job fields: title, company, location, job type (Full-time / Internship / Part-time), description, salary range, required skills, qualifications, eligibility criteria (min CGPA, min 10th/12th %), vacancies, application deadline.
- **Active / Closed** job status.

5.4 Job Listing & Eligibility Engine

- Students see a feed of all active jobs.
- **Automatic eligibility check** on the server side — if a student's CGPA/percentages fall below the job's requirements, they are warned and the apply button is disabled.
- **Save / Unsave** jobs (bookmark feature).
- **Job detail page** with full description, eligibility, deadline countdown, and a one-click apply flow.
- **Unique application** enforced at the database level (composite unique index on `job + student`).

5.5 Application Lifecycle

Status flow: **Pending** → **Shortlisted** → **Accepted / Rejected** - Students submit applications with their profile as the resume snapshot. - Officers review applicants per job, filter by status, and bulk-update. - Every status change triggers a **real-time notification** to the student. - Students can **withdraw** a pending application.

5.6 Real-Time Notifications (Socket.IO)

- Per-user socket rooms — every notification is pushed live without page refresh.
- Triggers: new job posted, application status updated, student verified/rejected, deadline reminder.
- A **Notification Center** page lists all notifications with read/unread state.
- In-app notification badge updates in real time.

5.7 Deadline Reminder System

- A **background scheduler** runs every 6 hours and sends reminders to students who have saved a job but not applied, at **3, 2, 1, and 0 days** before the deadline.
- An **on-login catch-up hook** runs the same check when a student logs in, so no reminders are missed even if the server has been idle (important for free-tier hosting that sleeps between requests).
- Reminders are sent both **via email** (Nodemailer) and **in-app** via Socket.IO.

5.8 Officer Dashboard & Student Management

- **Verify Students** page: pending students are listed and can be approved or rejected with a reason.

- **All Students** directory: searchable, filterable list of all students with derived columns (Placement Status: Placed / Trying / Not Trying, total/accepted/in-progress/rejected counts) computed **server-side** in a single bulk query to avoid N+1 round-trips.
- **Placement Status filter** on the directory (Placed, Trying, Not Trying) so the officer can quickly find unplaced students.
- **Create student account** form with auto-generated temp password.

5.9 Job Management (Officer)

- **Manage Jobs** page: list all jobs with applicant counts, edit/close/delete, view applicants per job.
- **Applicant review** per job: see all applicants with profile snapshot, current status, and shortlist/accept/reject actions.

5.10 Analytics & Statistics

- **Student Dashboard:** personal stats — total applications, accepted, in progress, rejected, profile completion %.
- **Officer Stats page:** department-level placement statistics (placed students, average package, company-wise breakdown).
- **Admin Dashboard:** system-wide overview — total students, verified, placed, total jobs, total applications, recent activity.

5.11 Admin Module

- Manage officer accounts (create, view, suspend).
- Full visibility into the platform with the master dashboard.

5.12 Email Service

- Nodemailer-based transactional emails for: registration OTP, application status updates, deadline reminders, account verification status.
- Emails sent through configurable SMTP (Gmail / SendGrid / etc. via env variables).

5.13 Security

- **Role-Based Access Control (RBAC)** middleware on every protected route.
- **Firestore Admin SDK** verification for Google sign-in tokens.
- **JWT** with expiry for API auth.
- **Password hashing** (bcrypt) for local accounts.
- **Sensitive fields excluded** from API responses (e.g. `tempPassword` is never returned after first view).
- **CORS** locked to the configured frontend origin.
- **10 MB JSON body limit** to prevent payload abuse.

6. Database Schema (Overview)

User — name, email, role (student/officer/admin), Firebase UID, profile (academic + personal), saved jobs, status, flags. **Job** — title, company, location, type, eligibility (min CGPA/10th/12th %), vacancies, deadline, status, postedBy. **Application** — job ref, student ref, status (pending/shortlisted/accepted/rejected), appliedAt. Unique index on (job, student). **Notification** — recipient, type, message, read flag, link, createdAt.

7. REST API Endpoints (Sample)

Method	Endpoint	Purpose
POST	/api/auth/register	Student registration
POST	/api/auth/login	Email/password login
POST	/api/auth/firebase	Firebase Google login
GET	/api/auth/me	Current user (with flags)
GET	/api/jobs	List active jobs (student)
POST	/api/jobs	Create job (officer)
POST	/api/applications	Apply to job
PATCH	/api/applications/:id/status	Update status (officer)
GET	/api/students	All students (officer)
GET	/api/officer/pending	Pending verifications
GET	/api/stats/dashboard	Role-based stats
GET	/api/notifications	User notifications

Full API documentation is available in [/docs/](#).

8. Folder Structure

```
college-placement-system/
├── backend/
│   ├── controllers/    # authController
│   ├── middleware/     # auth (JWT + role)
│   ├── models/         # User, Job, Application, Notification
│   └── routes/         # auth, jobs, applications, students, officer, admin, stats, ...
```

```

├── services/      # emailService, deadlineReminders
├── server.js
├── seed.js
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── auth/      # Login, Register, AdminOfficerLogin
│   │   │   ├── student/   # Dashboard, Jobs, JobDetails, Applications, Profile, No
│   │   │   ├── officer/   # Dashboard, VerifyStudents, ManageJobs, AllStudents, St
│   │   │   └── admin/     # Dashboard
│   │   ├── context/      # AuthContext, SocketContext
│   │   ├── firebase/     # Firebase config
│   │   ├── lib/          # api client, utils
│   │   ├── routes/       # ProtectedRoute, role guards
│   │   ├── App.jsx
│   │   └── main.jsx
│   └── package.json
└── docs/

```

9. How to Run Locally

Backend

```

cd backend
npm install
# configure .env: MONGODB_URI, JWT_SECRET, FRONTEND_URL, SMTP, FIREBASE_*
npm run seed      # optional: seed sample data
npm start         # runs on http://localhost:5000

```

Frontend

```

cd frontend
npm install
# configure .env: VITE_API_URL, VITE_FIREBASE_*
npm run dev       # runs on http://localhost:5173

```

10. Conclusion

The Smart Placement System successfully delivers a **complete, role-aware, real-time placement management platform** with the following highlights:

- ✓ Three role-based dashboards (Student / Officer / Admin)
- ✓ End-to-end job lifecycle (post → apply → shortlist → accept/reject)
- ✓ Server-side eligibility engine (CGPA, 10th/12th %)

-  Real-time notifications via Socket.IO
-  Automated deadline reminder system (in-app + email)
-  Officer-driven student onboarding with secure temp-password flow
-  Analytics dashboards for every role
-  Firebase + email/password hybrid authentication
-  Responsive UI with Tailwind CSS

The project demonstrates practical use of the **MERN stack, WebSockets, third-party auth, automated background jobs, and role-based security** — all essential building blocks of a real-world web application.